RELIABLE UDP USING ARQ PROTOCOLS

Computer Networks Laboratory Project Report

Institution	KIIT
Department	Computer Science and Engineering
Faculty Name	Vishal Meena
Student / Team Member	Swarnadeep Rakshit — 24051297
	Surisetty Sai Akhilesh — 24051746
	Swastik Dhople — 24051670
	Sunetra Das — 2405391
Section	CSE 23
Submission Date	28 October 2026

Project focus: reliable file transfer over UDP using Stop-and-Wait, Go-Back-N, and Selective Repeat ARQ.

This report includes the supplied terminal screenshots as project evidence.

Abstract

UDP is a connectionless transport-layer protocol that does not itself guarantee delivery, ordering, or retransmission. This project explores reliable file transfer over UDP using Automatic Repeat reQuest (ARQ) protocols: Stop-and-Wait, Go-Back-N, and Selective Repeat. A channel emulator is used to simulate network impairments. File integrity is checked using SHA-256 hashes, while transfer time and goodput are used to evaluate Selective Repeat performance.

1. Introduction

UDP provides a lightweight transport service, but it does not guarantee that every packet reaches the destination. ARQ protocols improve reliability through acknowledgements and retransmissions. This project implements three ARQ strategies and tests their file-transfer behavior.

2. Objectives

- Implement reliable file transfer over UDP.
- Study Stop-and-Wait, Go-Back-N, and Selective Repeat ARQ.
- Simulate packet loss and observe acknowledgement/retransmission behavior.
- Verify received file integrity using SHA-256 hashes.
- Measure transfer time and goodput at different simulated loss rates.

3. Tools and Technologies

Python, UDP sockets, Git, GitHub, a project channel emulator, PowerShell, CSV, and Excel.

4. Protocol Overview

4.1 Stop-and-Wait ARQ

The sender sends one packet and waits for its acknowledgement before sending the next. If an acknowledgement is not received within the timeout, the packet is retransmitted. This is simple but can have low link utilization.

4.2 Go-Back-N ARQ

The sender may transmit multiple packets within a sliding window. If a packet is lost, the sender retransmits the missing packet and subsequent outstanding packets as required by the protocol. This improves pipelining but may cause extra retransmissions.

4.3 Selective Repeat ARQ

The sender tracks outstanding packets and selectively retransmits packets that need recovery. The receiver can buffer out-of-order packets and acknowledge packets individually. This can reduce unnecessary retransmissions but requires more buffering and state management.

5. System Design and Implementation

The project uses UDP sockets for communication and shared packet helpers from common.py. The packet format uses the magic value RDT1, a packet type, sequence/acknowledgement fields, and a checksum. The project defines DATA, ACK, and END packet types. The channel emulator simulates network impairment such as loss and corruption.

6. Experimental Setup

The Selective Repeat performance tests use a 1 MiB test file (1,048,576 bytes), a window size of 4, and random seed 1. The recorded loss settings are 0%, 10%, 20%, and 30%. Sender and receiver are run in separate terminals. File integrity is checked by comparing the original and received SHA-256 hashes.

7. Performance Metrics

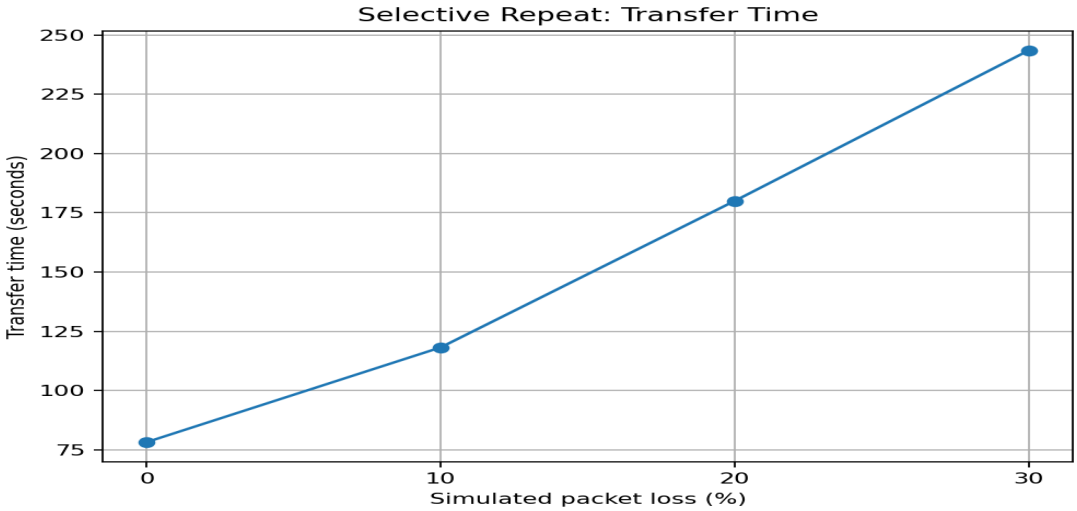
Transfer time: elapsed time required to complete a file transfer.
Goodput: useful file data transferred per second.

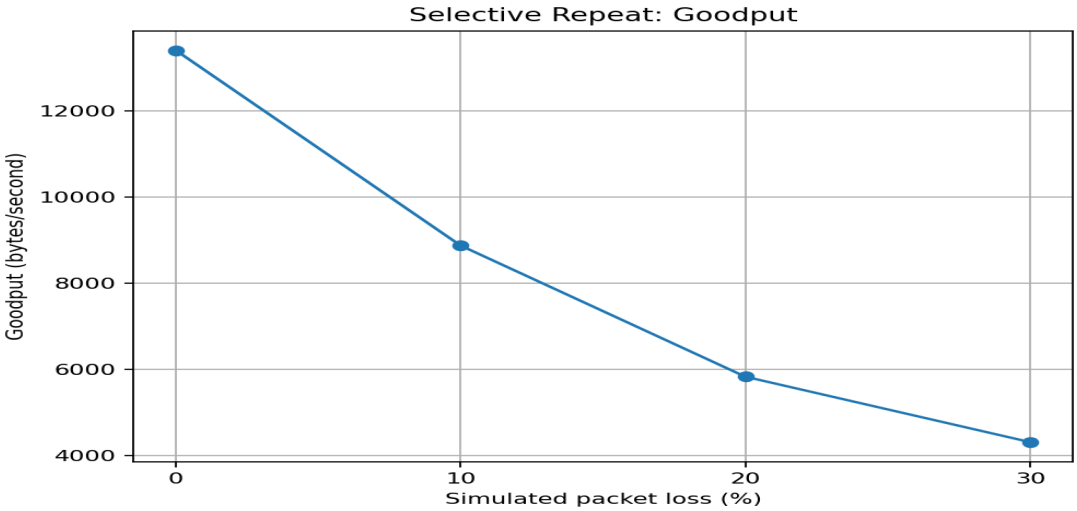
Goodput = File size (bytes) ÷ Transfer time (seconds)

8. Experimental Results

The table below summarizes the recorded Selective Repeat results from the project's working analysis. Transfer time can vary between runs because of simulated loss, retransmissions, timeouts, and system scheduling. These are recorded observations, not guaranteed repeatable timings.

Loss rate	Transfer time (s)	Goodput (B/s)
0%	78.206	13,407.87
10%	118.179	8,872.78
20%	180.000	5,825.42
30%	243.531	4,305.72





9. Evidence: Terminal Output and File Integrity

The following supplied screenshots show the Selective Repeat transfer and file-integrity check. The source-file and received-file SHA-256 values shown in the screenshots match.

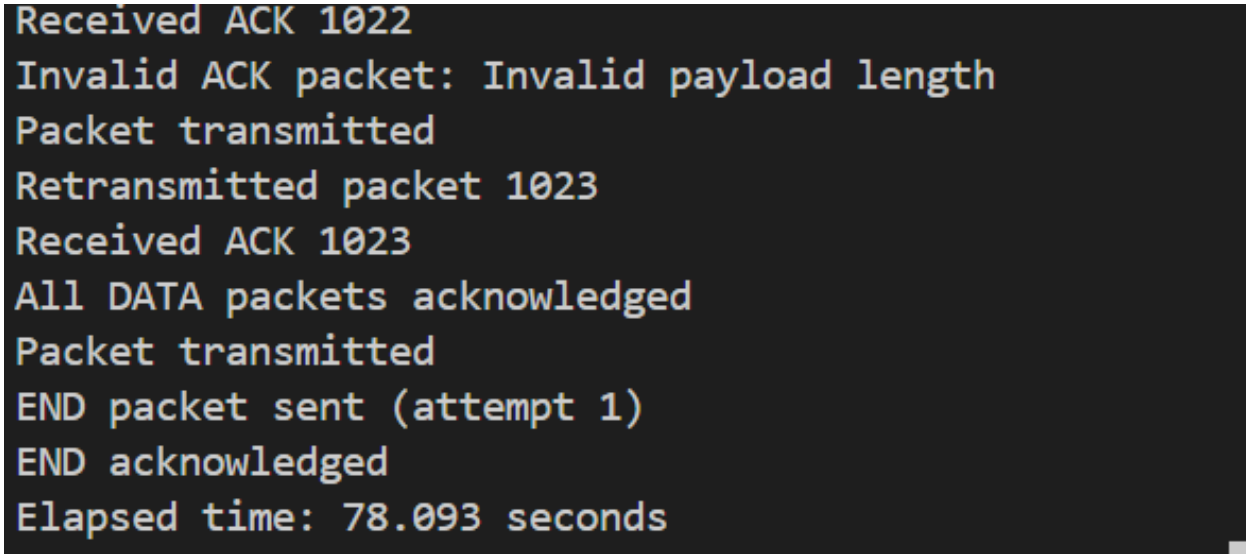


Figure 1. Selective Repeat sender output showing packet acknowledgements, retransmission, END acknowledgement, and elapsed time.

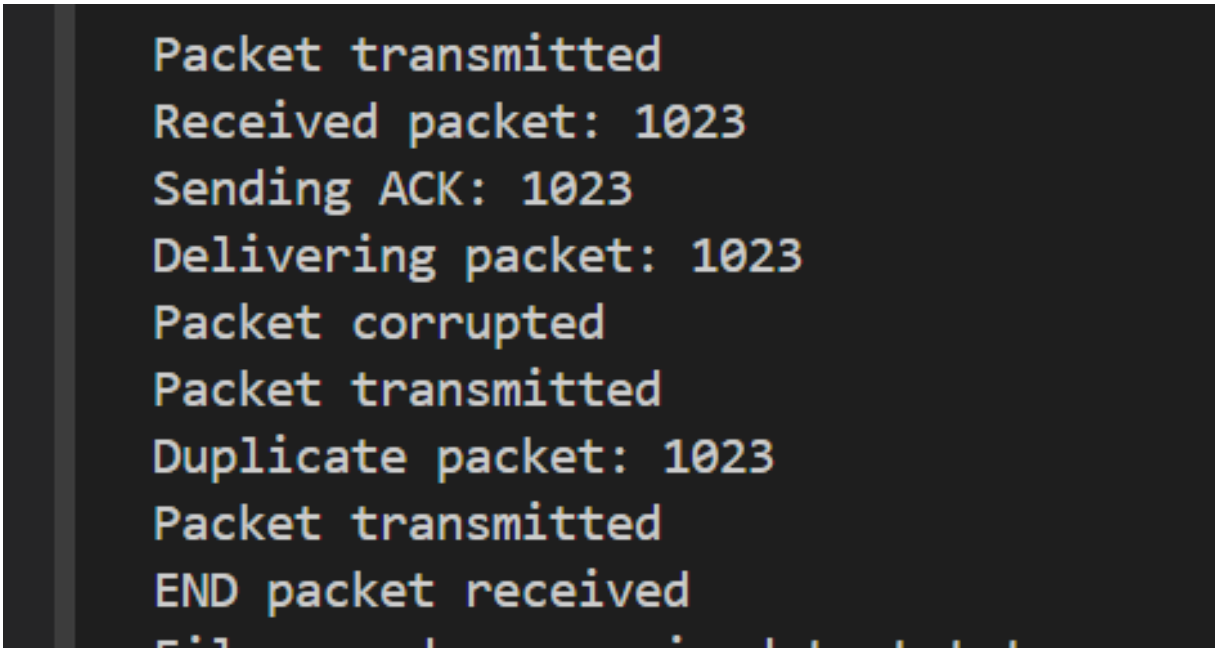


Figure 2. Receiver output showing packet reception, ACK transmission, corruption/duplication events, and END handling.



Figure 3. SHA-256 output for the source file.

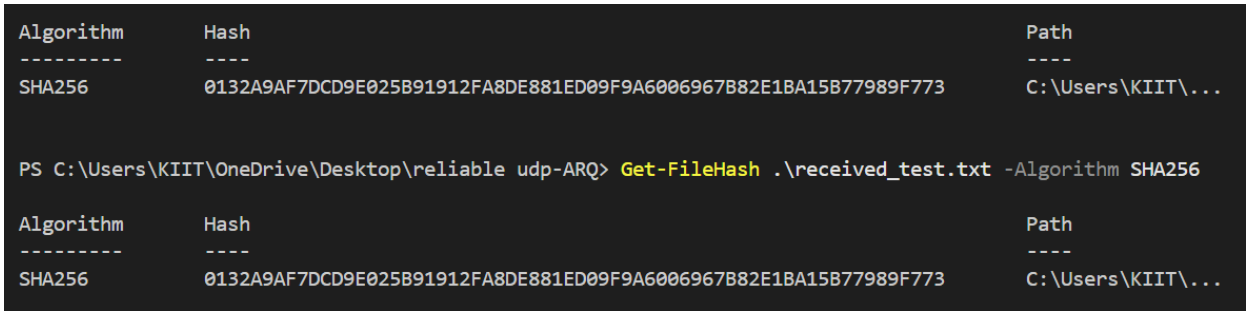


Figure 4. SHA-256 output for the received file; the displayed hash matches the source hash.

10. Discussion

The recorded results show increasing transfer time and decreasing goodput as simulated packet loss rises. This is consistent with additional recovery work when packets or acknowledgements are lost. Exact timing can vary across runs, so results should be interpreted as observations from the recorded experiments.

11. Limitations

- Simulated loss does not represent every characteristic of a real network.
- Transfer time varies between runs.
- Further repeated trials could provide more robust averages.
- Acknowledgement and END-packet behavior should be interpreted according to the actual implementation.

12. Conclusion

The ReliableUDP-ARQ project demonstrates how ARQ protocols can improve reliability of file transfer over UDP. Stop-and-Wait uses a simple one-packet-at-a-time method, Go-Back-N enables pipelining but may retransmit multiple packets, and Selective Repeat can retransmit selectively. The experiments illustrate how simulated packet loss can affect transfer time and goodput.

13. Future Enhancements

- Repeat each experiment and report averages and variability.
- Record retransmission counts and timeout events.
- Improve and document packet reordering and acknowledgement-loss simulation.
- Compare all three protocols under identical conditions.